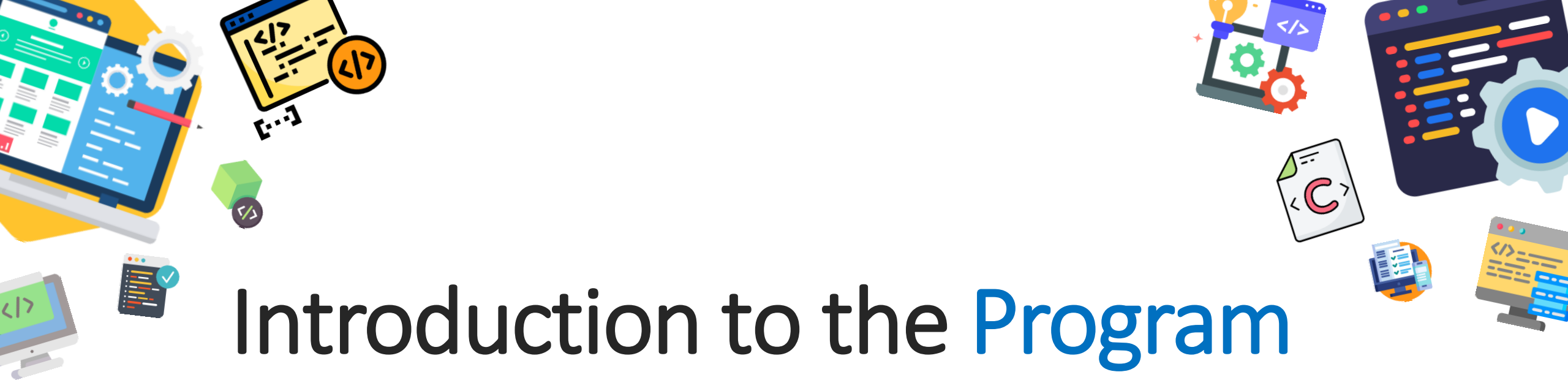




# Introduction to the Program Design and Programming

ICT 1305

---

CHAPTER 02

J.M.D.S.JAYASINGHE

Department Of ICT

Faculty Of Technology

Rajarata University Of Sri Lanka



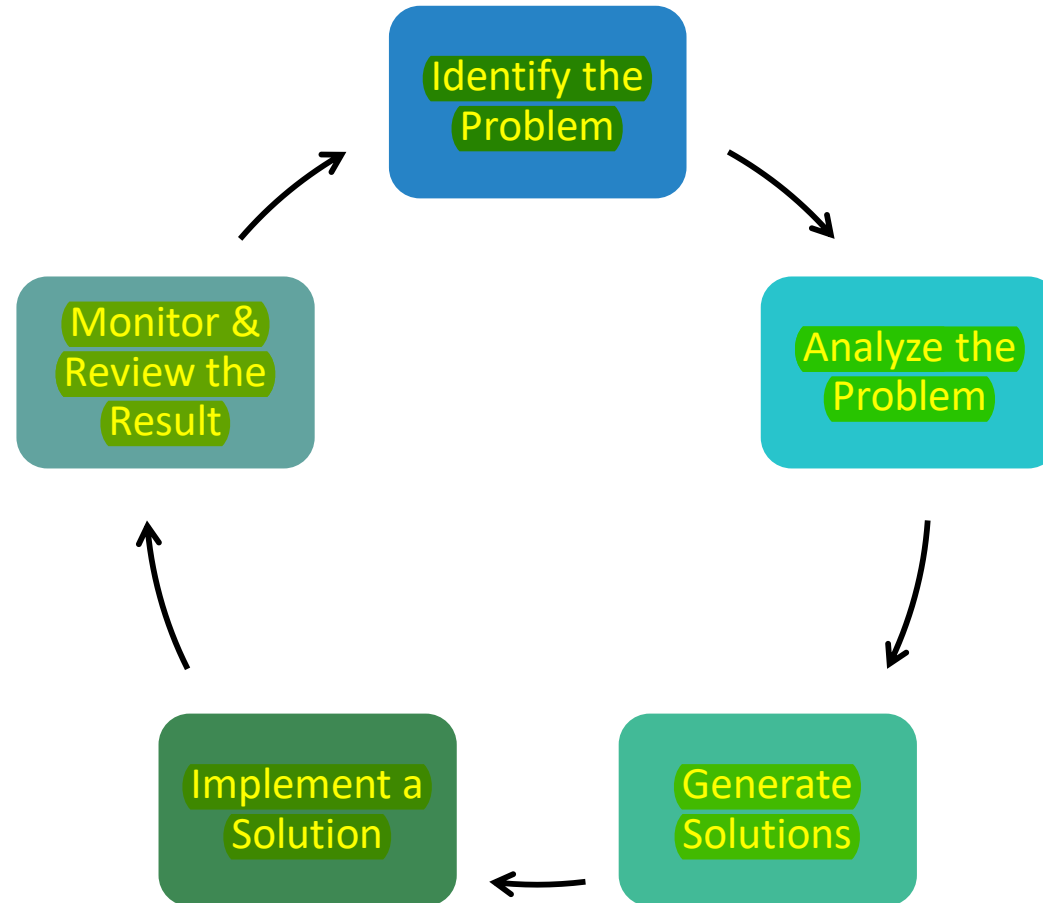
# Problem Solving

---

- Problem-solving is the process of identifying a challenge or issue, understanding its nature, and devising and implementing an effective solution.
- It is a critical skill used in various aspects of life, including personal decision-making, professional settings, and academic pursuits.

# Phases of Problem Solving

---



# Phases of Problem Solving

---

## Identify the Problem

- Clearly identify and understand the problem.
- Gather necessary data and feedback.

## Analyze the Problem

- Investigate the underlying root cause of the problem.
- Use tools like 5 Whys or Fishbone Diagram to analyze the root cause.
- Break the problem into smaller, manageable components.

# Phases of Problem Solving

---

## Generate Solutions

- Generate a variety of possible solutions.
- Encourage creative thinking for the same.

## Implement a Solution

- Evaluate all the possible solutions.
- Select the most feasible solution.
- Execute it with a structured plan.

# Phases of Problem Solving

---

## Monitor & Review the Result

- Track the effectiveness of the solution.
- Make changes if necessary to ensure success.

# Problem Solving using Computer

---

- Problem solving using computers refers to utilizing computational tools, methods, and programming techniques to analyze, design, and implement solutions for various problems.
- This process combines human logic with the computational power of machines to tackle complex tasks effectively and efficiently.
- After its creation, the software should be error free and well documented.
- Software development is the process of creating such software, which satisfies end user's requirements and needs.

# Phases of Problem Solving using Computer

---

The following six steps must be followed to solve a problem using computer,

1. Identify the Problem
2. Problem Analysis
3. Program Design - Algorithm, Flowchart and Pseudo code
4. Coding
5. 

sampadanaya and kriyathmaka kirima

 Compilation and Execution
6. Debugging and Testing



# Problem Analysis

---

- Problem analysis is the process of defining a problem and decomposing overall system into smaller parts to identify possible inputs, processes and outputs associated with the problem.
- This task is further subdivided into six subtasks namely:
  - a. Specifying the Objective
  - b. Specifying the Output
  - c. Specifying Input Requirements
  - d. Specifying Processing Requirements
  - e. Evaluating the Feasibility agama and shakyathawa
  - f. Problem Analysis Documentation

# Problem Analysis

---

## Specifying the Objective :

To solve a problem effectively, it's essential to clearly define it. Simple problems can be stated easily, allowing a quick transition to program design. However, complex problems involving multiple subsystems require thorough analysis, extensive research, and careful coordination of people, processes, and programs.

## Specifying the Output :

To determine system inputs, first identify the desired outputs. Designing output forms and formats helps specify results clearly. End users, who benefit from the software, are best suited to evaluate these forms. Programmers can create various designs, which must be assessed for usefulness.

# Problem Analysis

---

## **Specifying Input Requirements :**

Once outputs are defined, the necessary inputs and data sources must be identified. For instance, in a student record program, inputs like name, address, and roll number may come from the students or their supervisor.

## **Specifying Processing Requirements :**

After defining outputs and inputs, the process for converting inputs into outputs must be specified. For replacing or supplementing an existing system, evaluate current procedures for potential improvements. If creating a new system, study similar systems to guide development.

# Problem Analysis

---

## **Evaluating the Feasibility :**

After completing the initial steps, evaluate the practicality and feasibility of the solution. For replacing an existing system, assess whether the proposed improvements outperform the current system or similar alternatives.

## **Problem Analysis Documentation :**

At the end of program analysis, document all progress, including program objectives, output and input specifications, processing requirements, and feasibility, to ensure clarity and reference for further development.

# Program Design

---

- The second stage of software development, program design, involves creating algorithms, flowcharts, and pseudo code to structure the program for user-friendliness, feasibility, and optimization. Using the top-down design approach, tasks are divided into smaller modules and sub-modules, simplifying the programming process into manageable, independent parts.
- In program design we are mainly interested in designing:
  - Algorithms
  - Flowcharts
  - Pseudo codes

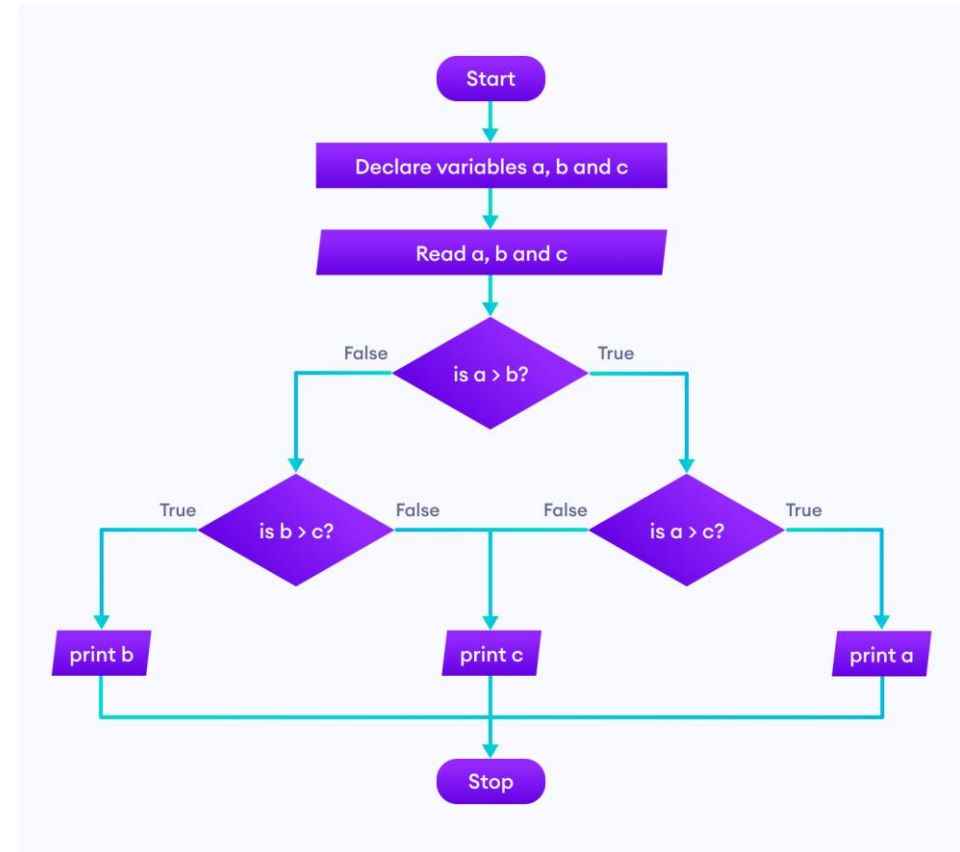
# Algorithms

---

- An algorithm is an effective step-by-step procedure for solving a problem.
- It is required to decide a solution before writing a program. The procedure of representing the solution in a natural language called an algorithm.
- We must design, develop and decide the final approach after a number of trials and errors, before actually writing the final code on an algorithm before we write the code.
- The same problem can be solved with different methods.
- It captures and refines all the aspects of the desired solution.

# Flowcharts

- Flowchart is basically a pictorial or diagrammatic representation of an algorithm using standard symbols.
- Flowchart is a graphical representation that explains the sequence of operations to be performed in order to solve a problem under consideration.



# Pseudo Codes

---

- Pseudo code is structured English for describing algorithms concisely.
- It is made up of two words, namely, pseudo meaning imitation and code meaning instructions.
- As the name suggests, pseudo code does not obey the syntax rules of any particular programming language
- i.e. it is not a real programming code. It allows the designer to focus on main logic without being distracted by programming languages syntax.
- Pseudo code is the intermediate state between an idea and its implementation(code) in a high-level language.



# Coding

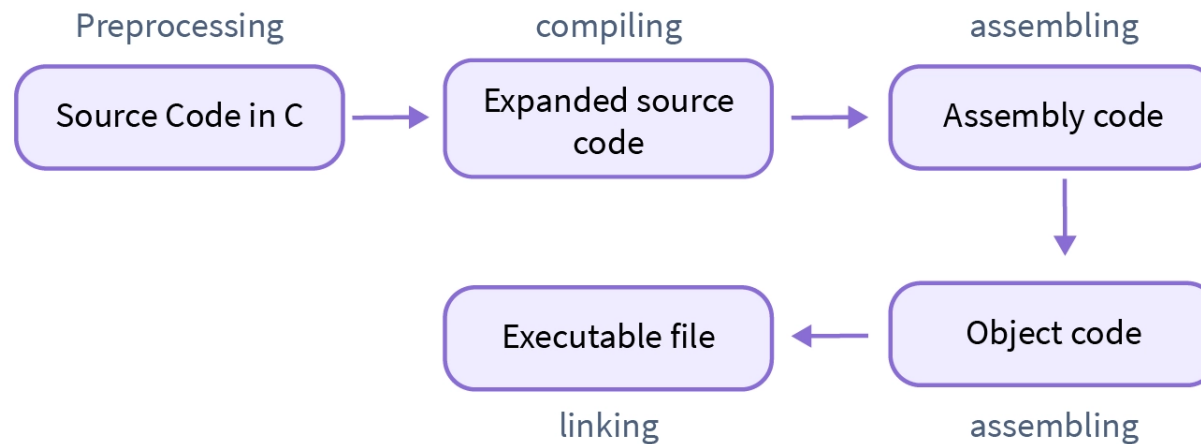
---

- In this stage, process of writing actual program takes place.
- A coded program is most popularly referred to as a source code.
- The coding process can be done in any language (high level and low level).
- The actual use of computer takes place in this stage in which the programmer writes a sequence of instructions ready for execution.
- Coding is also known as programming.

# Compilation and Execution

---

- Compilation is the process of converting the source code written by a programmer (in a high-level language like C, Java, or C++) into machine code or an intermediate form that the computer's processor can understand and execute. This process is carried out by a compiler.
- Execution is the process of running the compiled code (or interpreted code) on a computer. It refers to the actual process of carrying out the instructions defined in your program.



# Debugging and Testing

---

## Error

- An error in programming refers to any issue or problem that prevents a program from functioning as expected.
- Errors can occur during different stages of development, such as during writing, compiling, or running the program.
- They can range from simple syntax mistakes to complex logic errors that produce incorrect results.

# Debugging and Testing

---

## Types of Error:

### Syntax Errors:

These occur when the code violates the **grammatical rules** of the programming language. Example: Missing a semicolon in C++ or forgetting to close parentheses in Python.

### Logical Errors:

These are bugs that cause the program to **run but produce incorrect or unintended results**. The program executes without crashing, but the output is wrong. Example: Incorrect calculation or using the wrong formula in a program.

### Runtime Errors:

These occur while the program is running, causing the program to **crash or behave unexpectedly**. Example: Dividing by zero, accessing a non-existent file, or using a variable that has not been initialized.

# Debugging and Testing

---

- The designed and developed program undergoes several rigorous tests based on various real-time parameters and the program undergoes various levels of simulations.
- It must meet the user's requirements, which have to respond with the required time. It should generate all expected outputs to all the possible inputs.
- The program should also undergo bug fixing and all possible exception handling. If it fails to show the possible results, it should be checked for logical errors.
- Industries follow some testing methods like system testing, component testing and acceptance testing while developing complex applications.
- The errors identified while testing are debugged or rectified and tested again until all errors are removed from the program.

# Thank You

---